Project – Low Level Design

on

Finance Support Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number

S.no	Student name	Enrollment Number
1	MIMANSA KASHYAP	EN22IT301054
2	HARSH GUPTA	EN24CA5030057
3	ADITYA BAIRAGI	EN24CA5030009
4	SANDHYA PATIDAR	EN24CA5030150
5	SHUBHANJALI ROY	EN24CA5030161
6	APARNA SHARMA	EN24CA5030023

Group Name: Group 12D12

Project Number: AAI-03

Industry Mentor Name: Mr.Ashwin Sir

University Mentor Name: Ajeet Sigh Rajput

Academic Year: 2024-2026

Table of Contents

	Page No.
1.Introduction	3
1.1 Scope of the document	3
1.2 Intended Audience	3
1.3 System Overview	4
1.4 System Architecture Diagram	4-5
2.System Design.	5
2.1 Application Design	5-8
2.2 Process Flow	8
2.3 Information Flow	9
2.4 Components Design	9
2.5 Key Design Considerations	10-11
2.6 API Catalogue	11-12
3.References	13

1.Introduction

The **Finance Support Triage Agent** is a production-grade intelligent system designed to automate and streamline the handling of finance-related communications such as customer queries, transaction disputes, invoice issues, and account-related concerns. In traditional systems, such requests are manually reviewed, categorized, and responded to, which leads to delays, inefficiencies, and increased operational costs. This project leverages **Large Language Models (LLMs)** along with modern frameworks like **Langchain /CrewAI** to enable real-time classification, entity extraction, and automated response generation .The system processes incoming text data (emails, chats, tickets), classifies them based on urgency and intent (e.g., urgent fraud issue vs general inquiry), extracts key entities such as transaction IDs, account numbers, and dates using **Named Entity Recognition (NER)**, and generates context-aware draft responses. This significantly improves response time, accuracy, and customer satisfaction while reducing human workload.

1.1Scope of the document

This document provides a comprehensive overview of the design and implementation of the Finance Support Triage Agent. It includes system architecture, UML diagrams (conceptually described), application design, process flow, component design, API catalog, and key design considerations. The document serves as a technical blueprint for developers, architects, and stakeholders involved in the system development and deployment.

1.2 Intended Audience

This document is intended for:

- Software Developers and Engineers
- AI/ML Engineers
- System Architects
- Project Managers
- Stakeholders in Finance/Support Operations

It is also useful for students and professionals preparing similar AI-based automation projects.

1.3. System Overview

The Finance Support Triage Agent is built as an AI-driven pipeline that integrates multiple modules:

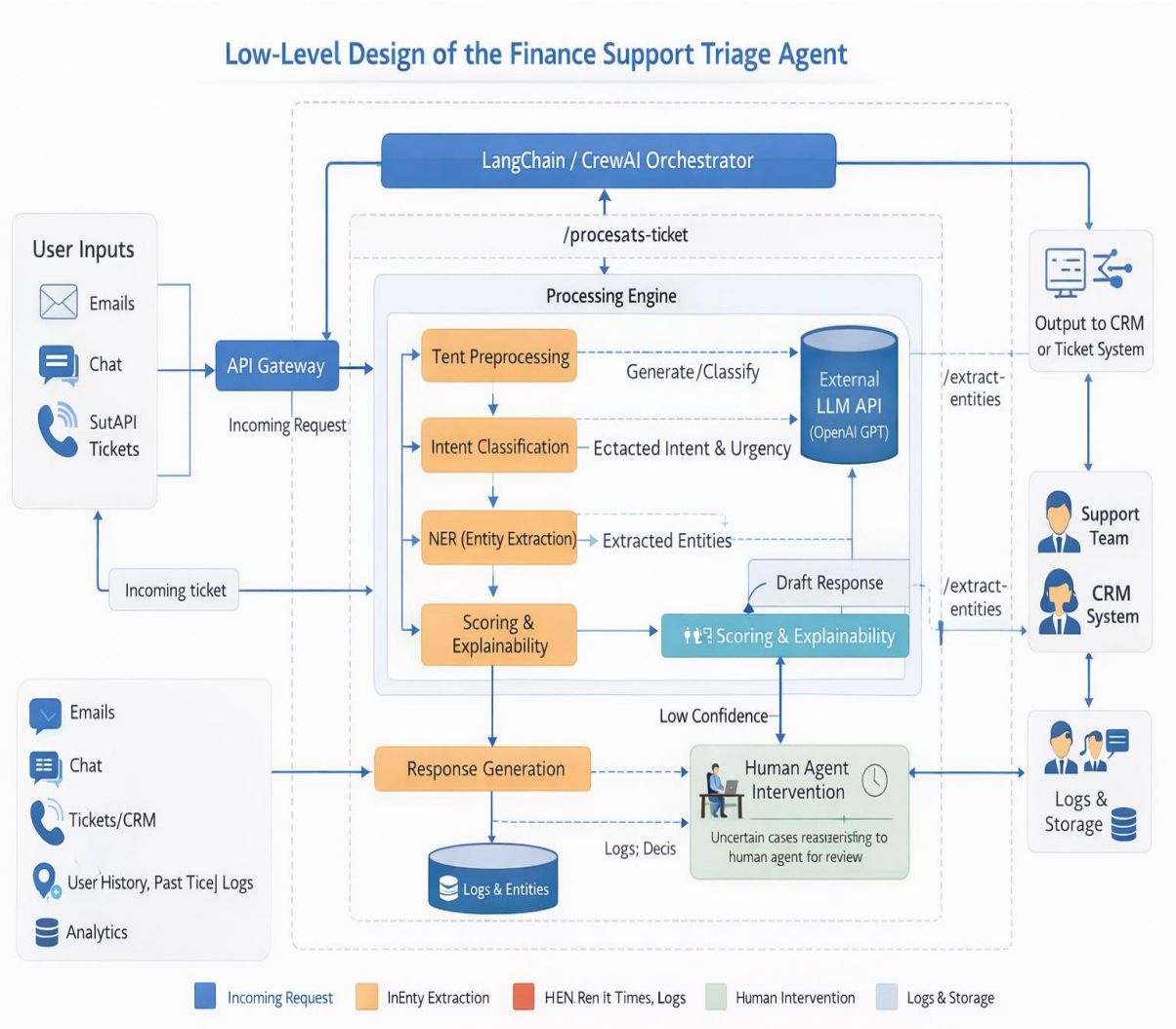
1. **Input Layer** – Accepts incoming communications (email, chat, tickets)
2. **Preprocessing Layer** – Cleans and normalizes text
3. **Classification Module** – Determines intent and urgency using LLM
4. **NER Module** – Extracts key financial entities
5. **Response Generator** – Creates draft replies using LLM API
6. **Routing Module** – Sends output to appropriate team or system

The system operates in real-time and can be integrated into existing CRM or ticketing platforms.

1.4. System Architecture Diagram

The architecture follows a **modular microservice-based design**:

- **Frontend / Input Interface**
- **API Gateway (FastAPI/Flask)**
- **Processing Engine**
  - Text Preprocessor
  - Classification Service (LLM)
  - NER Service
  - Response Generator
- **Orchestration Layer (LangChain / CrewAI Agents)**
- **Database (Logs, Tickets, Entities)**
- **External LLM API (e.g., OpenAI GPT)**



2.System Design

2.1 Application Design

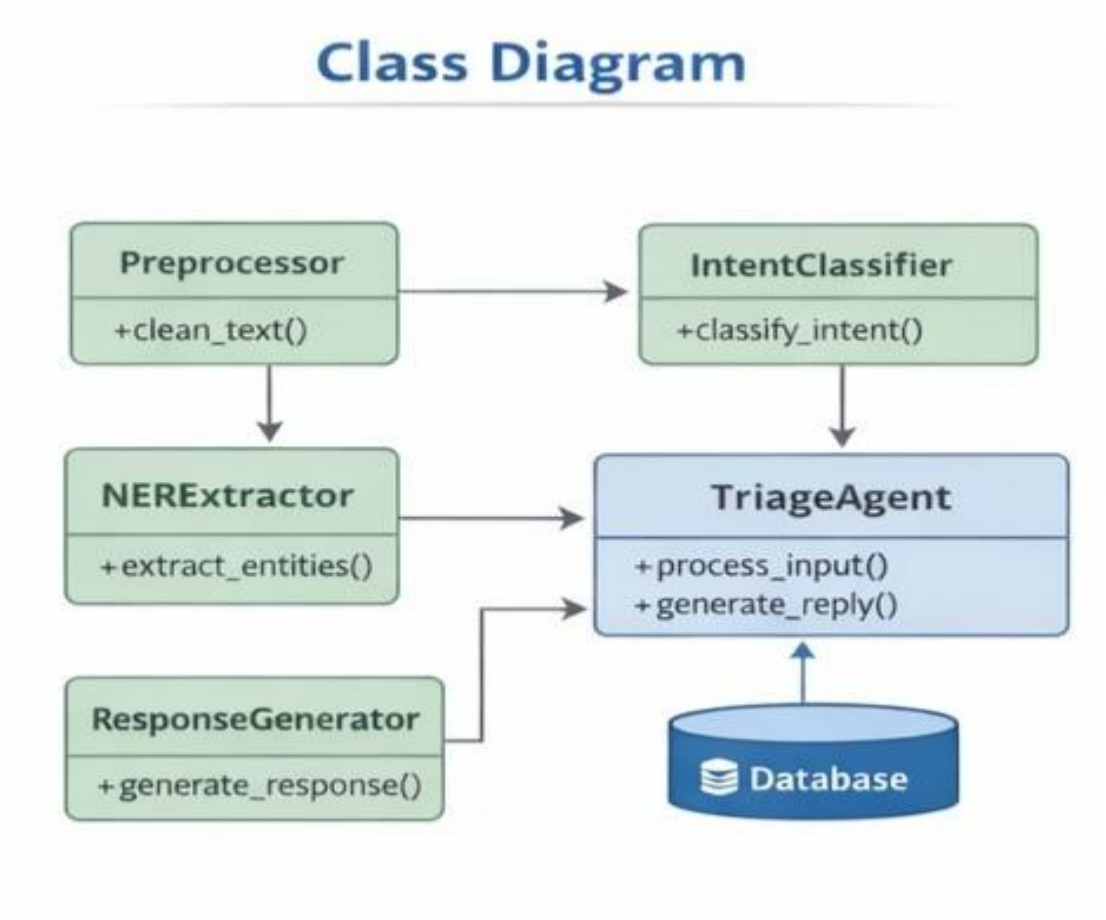
The application follows a layered architecture:

- **Presentation Layer** – Handles user input/output
- **Service Layer** – Business logic (classification, NER, response generation)
- **Integration Layer** – LLM API and external services
- **Data Layer** – Stores logs, extracted entities, and responses

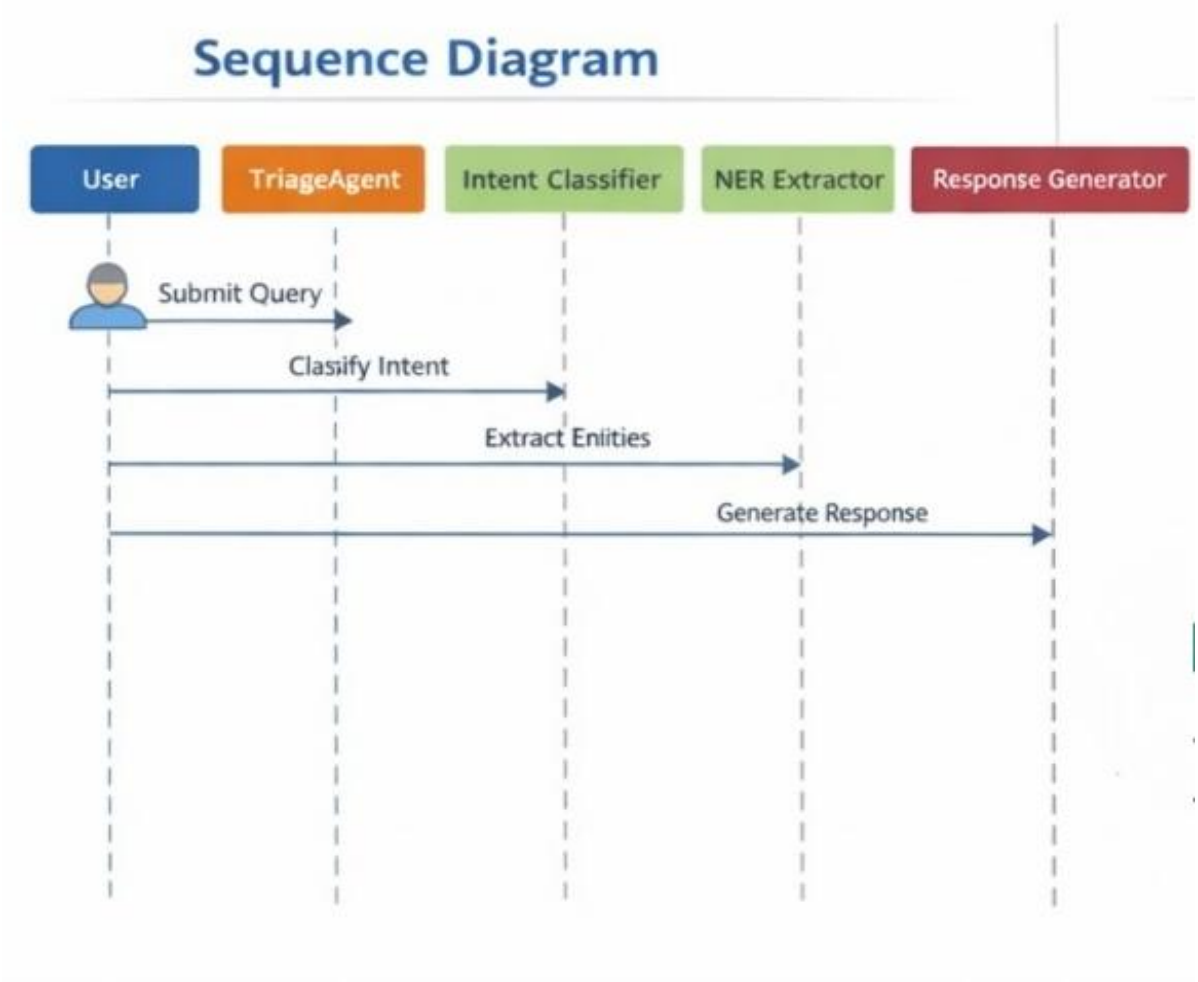
The system uses **LangChain/CrewAI** to orchestrate multiple AI agents:

- Classifier Agent
- NER Agent
- Response Generator Agent

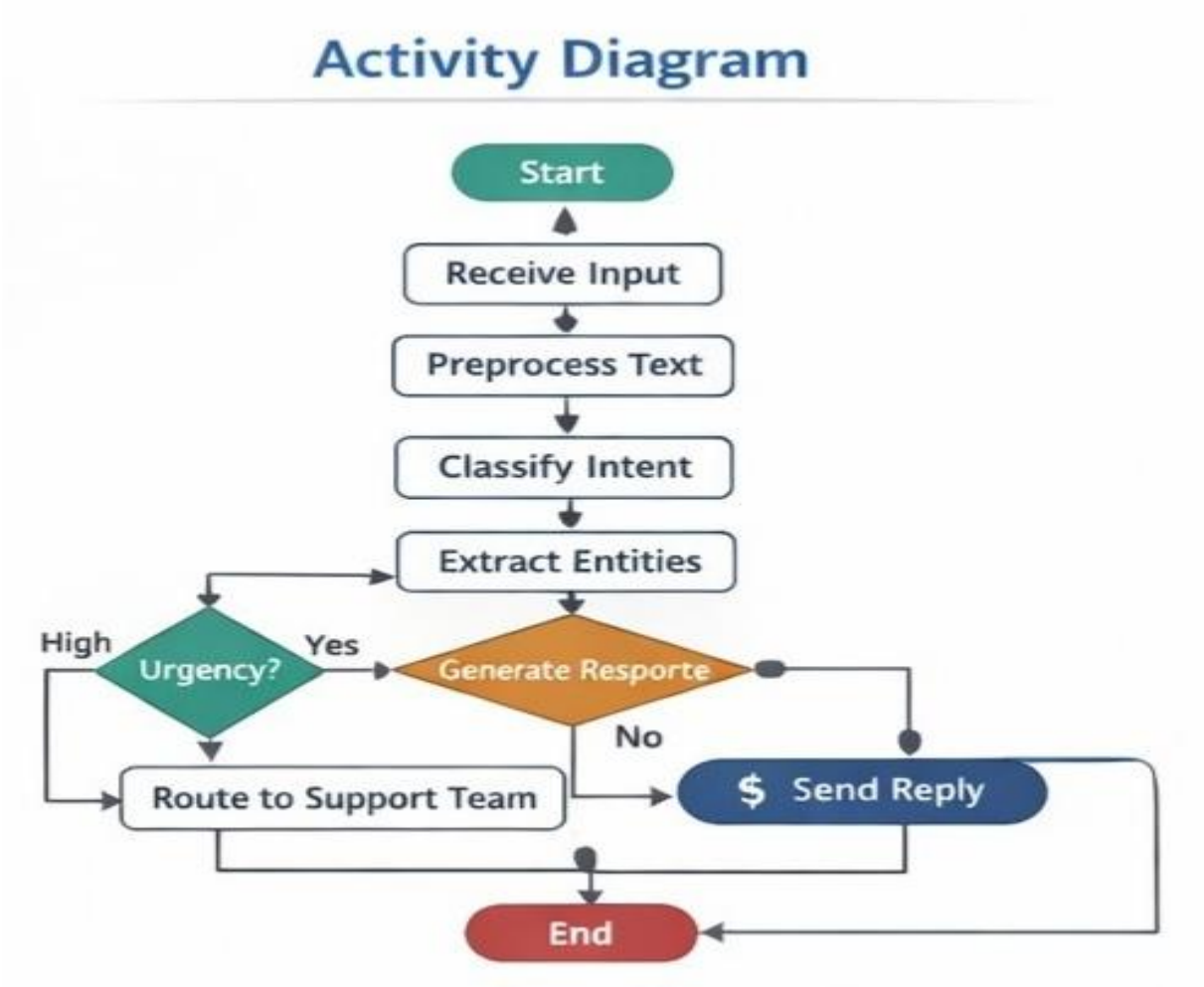
2.1.1 Class diagram



2.1.2 Sequence Diagram



2.1.3 Activity Diagram



2.2 Process Flow

- User submits a finance-related query
- Text is cleaned and preprocessed
- LLM classifies intent (e.g., complaint, query, fraud)
- Urgency level is assigned (High/Medium/Low)
- NER extracts entities (IDs, dates, amounts)
- Response is generated using LLM
- Ticket is routed or response is sent

2.3 Information Flow

- **Input Data:** Raw text (email/chat)
- **Processed Data:** Cleaned text
- **Extracted Data:** Entities (IDs, dates)
- **Output Data:** Classified intent + generated response

Flow ensures minimal data loss and high accuracy using structured pipelines.

2.4 Components Design

1. Preprocessing Module

- Removes noise, stopwords
- Standardizes input

2. Classification Module

- Uses LLM for intent detection
- Example: "Refund request" → Billing Issue

3. NER Module

- Extracts:
  - Transaction IDs
  - Dates
  - Account Numbers

4. Response Generator

- Uses prompt engineering
- Generates human-like replies

5. Orchestration Layer

- Managed using LangChain/CrewAI
- Coordinates agents efficiently

6. Database

- Stores logs, results, analytics

2.5 Key Design Considerations

The design of the Finance Support Triage Agent focuses on building a robust, scalable, and intelligent system capable of handling real-time financial communications efficiently. One of the primary considerations is **scalability**, achieved through a microservices-based architecture that allows independent components such as classification, NER, and response generation to scale horizontally based on workload. **Accuracy** is another critical factor, ensured by leveraging advanced Large Language Models (LLMs) with carefully engineered and fine-tuned prompts, enabling precise intent detection and reliable entity extraction. The system also prioritizes **low latency**, optimizing API calls and processing pipelines to deliver near real-time responses, which is essential in finance-related queries where timely action is crucial.

Scalability

Scalability ensures that the system can handle increasing volumes of requests without performance degradation. By using microservices architecture, each module (like classification or NER) can be scaled independently based on demand.

Accuracy

Accuracy is achieved using powerful LLMs and well-designed prompts. It ensures correct intent classification and precise extraction of entities such as transaction IDs, dates, and amounts.

Latency

Latency refers to the response time of the system. Optimized API calls, efficient pipelines, and minimal processing delays ensure quick responses, especially important for urgent financial issues.

Security

Security focuses on protecting sensitive financial data. This includes data encryption, secure API communication, authentication, and compliance with data protection standards.

Explainability

Explainability ensures that the system's decisions are transparent and understandable. Logs, intermediate outputs, and structured responses help in debugging and trust-building.

2.6 API Catalogue

The Finance Support Triage Agent exposes a set of RESTful APIs to handle various functionalities such as content generation, template retrieval, user history tracking, and data storage. These APIs enable seamless integration with external systems like CRM platforms, chat applications, and support dashboards.

1. Generate Content API

This API generates context-aware responses for finance-related queries using the LLM. It takes user input along with intent and urgency and returns a suitable reply.

Endpoint: `/generate-content`
Method: POST

Example:

```
{ "query": "Refund issue", "intent": "Refund",
"urgency": "High" }
```

2. Get Templates API

This API retrieves predefined templates for common financial queries like refunds or fraud cases. It helps maintain consistency and reduces LLM usage.

Endpoint: `/get-templates`
Method: GET

3. User History API

This API fetches past user interactions from the database to provide context-aware responses.

Endpoint: `/user-history`
Method: GET

Example:

```
{ "user_id": "U101" }
```

4. Store Content API

This API stores user queries, responses, and extracted entities in the database for logging and analysis.

Endpoint: `/store-content`
Method: POST

5. NER (Entity Extraction) API

This API extracts important entities like transaction IDs and dates from user input using NLP/LLM.

Endpoint: `/extract-entities`
Method: POST

Example:

```
{ "text": "Transaction ID 12345" }
```

6. System Orchestration API

This is the main API that manages the complete workflow using LangChain/CrewAI. It performs classification, entity extraction, and response generation in one pipeline.

Endpoint: `/process-ticket`
Method: POST

3. References

- **LangChain Documentation**
  https://docs.langchain.com/
  → Used for building LLM-based pipelines and chaining tasks like classification, NER, and response generation.
- **CrewAI Documentation**
  https://docs.crewai.com/
  → Used for multi-agent orchestration and workflow automation.
- **OpenAI API Documentation**
  https://platform.openai.com/docs/
  → Used for integrating Large Language Models (LLMs) for text classification and generation.
- **FastAPI Documentation**
  https://fastapi.tiangolo.com/
  → Used for building high-performance APIs.
- **Flask Documentation**
  https://flask.palletsprojects.com/
  → Alternative lightweight framework for API development.